Modular Visual Question Answering via Code Generation

Sanjay Subramanian[1] **Medhini Narasimhan**[1] **Kushal Khangaonkar**[1]
Kevin Yang[1] **Arsha Nagrani**[2] **Cordelia Schmid**[2] **Andy Zeng**[2]
Trevor Darrell[1] **Dan Klein**[1]
[1]UC Berkeley [2]Google Research
{sanjayss,medhini,kushaltk,yangk,trevordarrell,klein}@berkeley.edu,
{anagrani,cordelias,andyzeng}@google.com

arXiv:2306.05392v1 [cs.CL] 8 Jun 2023

Abstract

We present a framework that formulates visual question answering as modular code generation. In contrast to prior work on modular approaches to VQA, our approach requires no additional training and relies on pre-trained language models (LMs), visual models pre-trained on image-caption pairs, and fifty VQA examples used for in-context learning. The generated Python programs invoke and compose the outputs of the visual models using arithmetic and conditional logic. Our approach improves accuracy on the COVR dataset by at least 3% and on the GQA dataset by roughly 2% compared to the few-shot baseline that does not employ code generation.

1 Introduction

The scope of reasoning needed for visual question answering (VQA) is vast, and demands the synthesis of many skills – from grounding language to pixels (Goyal et al., 2017; Radford et al., 2021; Zhai et al., 2022) and spatial reasoning (Hudson and Manning, 2019) to commonsense and knowledge-based reasoning (Marino et al., 2019). Consider the question *"Is the carriage to the right of a horse?"*. To consistently answer such questions correctly, a system must recognize that the question is the conjunction of two subquestions: *"Is there a horse?"* and *"Is the carriage to the right of the horse?"* Scaling the typical finetuning paradigm to all possible combinations of reasoning skills is prohibitively expensive in annotation cost and makes it difficult to add skills to an already-trained system.

Modular approaches, on the other hand – from classic methods (Krishnamurthy and Kollar, 2013), to differentiable neural module networks (NMNs) (Andreas et al., 2016; Hu et al., 2017; Saqur and Narasimhan, 2020)) – offer a potential route to leverage and scale to the compositional nature of visual reasoning as a means to generalize: i.e., *infinite use of finite means*. However, the modules of an NMN must still be trained jointly on a large dataset, and are also restricted in that they (i) require a parser, which must be modified if modules are added or removed from the system, and (ii) require retraining if a module is replaced.

In this work, we investigate an alternative class of modular VQA approaches, whereby building on the recent advent of highly capable out-of-the-box language models (LMs) (Chen et al., 2021; Ouyang et al., 2022) and visual language models (VLMs) (Li et al., 2022), we develop systems that formulate VQA as a program synthesis problem. Specifically, our method `CodeVQA`, illustrated in Figure 1, uses code-writing LMs to take questions as input, and outputs code to (i) orchestrate a series of visual primitive APIs that wrap around VLMs to probe the image for specific pieces of visual information (e.g., captions, pixel locations of entities, or image-text similarity scores), and (ii) reason about that information with the full expression of Python code (e.g. arithmetic, logic structures, feedback loops, etc.) to arrive at an answer. From a practical perspective, the modularity of `CodeVQA` combined with the few-shot prompting capabilities of LMs enable it to adapt to a broad range of desired VQA label distributions without additional model training, and benefits from replacing individual modules with improved versions as they become available.

We evaluate `CodeVQA` in the few-shot VQA setting, which has seen a great deal of recent work (Alayrac et al., 2022; Jin et al., 2021; Yang et al., 2021; Tiong et al., 2022). Our method outperforms previous approaches by at least 3% on the COVR dataset (Bogin et al., 2021), which requires reasoning over multiple images, and by roughly 2% on the GQA dataset (Hudson and Manning, 2019). Our results suggest that the benefits of modularity with recent off-the-shelf models can be realized in VQA without additional model training.[1]

[1]Our code and annotated programs are available at https://github.com/sanjayss34/codevqa.

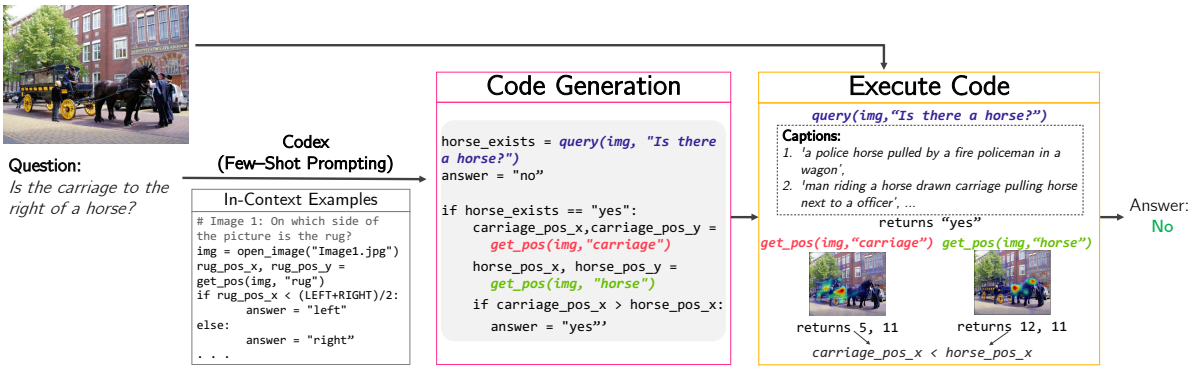

Figure 1: **CodeVQA Overview.** CodeVQA first prompts Codex with in-context examples that break down a given question into Python code. Using just the question, Codex generates an executable program that composes pre-defined visual modules using conditional logic, arithmetic, etc. The visual module, query answers a question by captioning the image and using an LM to answer based on the captions. get_pos retrieves the location of the object. Here, CodeVQA correctly identifies the question as a conjunction of a query and a spatial comparison and arrives at the right answer.

2 Related Work

Several recent approaches for reasoning tasks consist of an LM that writes programs and an interpreter for these programs. Liang et al. (2022) applies this approach to robotics. Cheng et al. (2023) introduces a framework for reasoning jointly over tables, text, and images, where the images are represented by image captions. Subramanian et al. (2022) used a syntactic parser and hard-coded rules rather than an LM to aggregate outputs from CLIP (Radford et al., 2021) for zero-shot referring expression comprehension; their finding that CLIP is not useful for spatial keywords motivates our code generation approach to spatial reasoning.

Concurrent with our work, other papers have introduced similar frameworks for multi-hop VQA (Gupta and Kembhavi, 2023; Surís et al., 2023). These papers conflate the benefit of program synthesis with the benefits of the LM, in-context examples, and vision models used as primitives. By contrast, we analyze the effect of program synthesis by comparing CodeVQA against a strong LM-based few-shot baseline using the same in-context example selection method. Moreover, while these frameworks rely on supervised VQA or object detection models, we show that we can obtain comparable performance (on the GQA dataset) using only the LM and models pre-trained on image-text pairs.

3 Few-shot VQA via Code Generation

In visual question answering (VQA), the inputs to the system are an image and a question and the output is a textual answer. We consider the few-shot VQA setting in which the system has access to only a small number (50) of human-annotated VQA instances.

Overview. Fig 1 illustrates our approach. Given an image and a corresponding question, CodeVQA first generates a Python program using just the question. It then executes this program, using the image when necessary, to predict the answer. We first define the set of code primitives that our system uses (§ 3.1). Then we describe how we generate a program that composes these primitives based on the question (§ 3.2). Finally, we enumerate the pre-trained models that we employ (§ 3.3).

3.1 Code Primitives

Primitives define basic operations over the image or over text that are often useful for VQA. In CodeVQA, we use three primitives, which are defined below. Each of these primitives is implemented using image-text matching (ITM), image-text contrastive (ITC), and image-captioning models, each of which can be trained with only image-caption pairs. The difference between ITM and ITC is that ITC computes separate image and text embeddings and takes a dot product, while ITM performs early fusion on the image and text features and is thus more computationally expensive. We note that our framework is not tied to this choice of primitives and can support other, more complex primitives that could draw on other aspects of the programming language and third-party libraries.

query(image, question) This function answers a question about the given image. Our implementation of this function is based on PnP-VQA (Tiong et al., 2022) and PICa (Yang et al., 2021) and is implemented with the following steps: (1) using the ITM model, compute the GradCAM (Sel-

varaju et al., 2016) between the question and the image (averaged over question tokens), (2) sample $K = 20$ image patches based on their GradCAM score, (3) generate a captions from the sampled patches using the captioning model, (4) Repeat steps (2) and (3) until C unique captions have been generated, and (5) predict the answer by prompting an LM with the question, captions, and in-context examples. The in-context examples in step (5) are selected as described in § 3.2. When the dataset involves reasoning over multiple images, each in-context example has the captions for all images.

`get_pos(image, text)` This function computes the GradCAM between the given text tokens and the image using the ITM model and returns the (x, y) pair that maximizes the GradCAM value. Note that this application of GradCAM is different from the one in `query` since we do not average over all question tokens. See Appendix B for more information on how we compute GradCAM maps.

`find_matching_image(images, text)` In the setting where multiple images are associated with each question, there are questions that refer specifically to one image (e.g. "What is the woman holding?"). This function can be used to select the most relevant image from the set. It is implemented by scoring each image with the text using the ITC model and picking the image with the highest score.

3.2 Code generation

In the first stage of `CodeVQA`, we generate a Python program based on the question. Using Python over a domain-specific language is advantageous because (1) it supports arithmetic as well as control flow including loops and if statements (Liang et al., 2022)–all of which we use in our programs–and (2) large LMs for code generation (e.g. Codex (Chen et al., 2021)) have been trained on a large amount of Python code.

We construct a prompt that consists of an instruction, constants that define the dimensions of the image, and import statements and API documentation (as a code comment) that specify the available functions. In addition to the prompt, the input to the LM also includes expert-annotated programs for several in-context examples. An in-context example for few-shot prompting on the COVR dataset is shown below (question in gray, the program is highlighted).

```
# Image Set 1: How many images contain exactly
2 pink shoes??
images = open_images("ImageSet1.jpg")
count = 0
for image in images:
    two_pink_shoes = query(image, "Are
        there exactly 2 pink shoes?")
    if two_pink_shoes == "yes":
        count += 1
answer = count
```

For an example of the rest of the prompt for the LM, see Appendix A. When executing the generated program results in a runtime error, we return call `query` on the image and the original question (including captions for all images if the instance involves multiple images).

Since all annotated programs cannot fit into a single input to the model, we must select which programs to use as in-context examples for each test question. Following Wang et al. (2022), we use sentence embeddings[2] to retrieve the most similar questions for each test question.

3.3 Component models

Our approach relies on four pre-trained models: a code generation model, an ITM model, an ITC model, an IC model, and a question-answering LM for answering questions based on captions. We use the `code-davinci-002` model (Chen et al., 2021) via the OpenAI API for both generating programs and for question-answering. We use the BLIP models (Li et al., 2022) finetuned for ITM, ITC, and captioning.

4 Experiments

4.1 Implementation Details

See Appendix C for implementation details.

4.2 Datasets

The GQA dataset (Hudson and Manning, 2019) contains multi-hop questions generated from human-annotated scene graphs of individual images in Visual Genome (Krishna et al., 2016). The COVR dataset (Bogin et al., 2021) contains multi-hop questions about *sets of images* in the Visual Genome and imSitu (Yatskar et al., 2016) datasets. These questions are synthetically generated from templates and are then paraphrased by humans. Unless otherwise specified, we present results on the paraphrased questions. The NLVR2 dataset (Suhr

[2] https://huggingface.co/sentence-transformers/all-mpnet-base-v2

Model	GQA Acc.	COVR Acc.	NLVR2 Acc.
Finetuned			
VisualBERT	–	57.9	67.0
VinVL-Base	65.1	–	83.1
Zero-shot			
FewVLM	29.3	–	–
PnP-VQA	42.3	–	–
BLIP-v2*	44.7	–	–
Few-shot			
FewVLM	35.7	–	–
VisProg*	50.5†	–	62.4
ViperGPT*	48.2	–	–
Few-shot PnP-VQA	46.6	45.8	63.4
CodeVQA (ours)	**49.0**	**50.7**	**64.0**

Table 1: **Results on GQA (testdev), COVR (test), and NLVR2 (test-public)** datasets from CodeVQA, Few-shot PnP-VQA, prior work VisualBERT (Li et al., 2019), VinVL-Base (Zhang et al., 2021), FewVLM (Jin et al., 2021), PnP-VQA (Tiong et al., 2022), and concurrent work (marked with *) BLIP-v2 (Li et al., 2023), VisProg (Gupta and Kembhavi, 2023), and ViperGPT (Surís et al., 2023). Our method outperforms all few-shot methods from prior work. Highest few-shot scores for each full dataset are in **bold**. †indicates an evaluation on a stratified sample of the test set, which may not be directly comparable to results on the full test set.

et al., 2019) contains statements about *pairs of images*, and the task is to determine whether each statement is true or false (we rephrase the statements as questions before feeding it to the methods that we evaluate). Appendix H has further details about the datasets. For each of the three datasets, we wrote programs for 50 questions randomly sampled from the corresponding training set. Unless stated otherwise, we put 12 in-context examples in a prompt for a single-image dataset and 6 in-context examples in a prompt for a multi-image dataset (since including captions for multiple images increases the necessary context size for each example). We report the exact-match accuracies of the lower-cased answers.

4.3 Baseline

Our primary baseline is an adaptation of PnP-VQA (Tiong et al., 2022) to the few-shot setting. We refer to it as "Few-shot PnP-VQA." This baseline is equivalent to running the five-step `query` procedure described in § 3.1 for every question. We also compare to zero-shot and few-shot methods from prior and concurrent work. Appendix D contains further details about those methods.

4.4 Results

Table 1 shows the results on the three datasets. CodeVQA has the highest accuracy among the few-shot techniques. Compared to Few-shot PnP-VQA, it has markedly better performance on COVR, which makes sense because in this dataset, the baseline approach must combine information across image captions for multiple images when given a single prompt. On the other hand, our method loops over the images and queries a single image at a time or selects the image most relevant to the question. Indeed, Table 3 shows that CodeVQA has the greatest advantage on instances involving 4 or 5 images. Compared to concurrent work that also uses program synthesis, CodeVQA generally performs better, which could be due to methodological differences like our in-context example retrieval or due to implementation details.

Fig. 2 shows a qualitative comparison of CodeVQA and the baseline Few-shot PnP-VQA on the COVR dataset. CodeVQA answers the question correctly by answering a simpler question for each image and comparing the answers, while Few-shot PnP-VQA answers incorrectly despite producing captions with the necessary information.

4.5 Ablations

Table 2 compares embedding-based retrieval of in-context examples with random retrieval. CodeVQA's improvement over Few-shot PnP-VQA is greater when in-context examples are retrieved by embedding. Embedding-based retrieval offers a systematic way to collect relevant in-context examples rather than curating a single set of examples as in Gupta and Kembhavi (2023).

In Appendix F, we include ablations for the question-answering LM and for the number of shots in the prompt as well as results on validation sets. Table 4 shows that CodeVQA improves over Few-shot PnP-VQA when either `code-davinci-002` or `text-davinci-003` is used as the question-answering LM. Table 5 shows roughly constant accuracy as the number of in-context examples is varied.

4.6 Analysis

Figure 3 breaks down accuracy by question type. CodeVQA's greatest improvement (roughly 30%) is in the subset consisting of questions about left/right or top/bottom object positions. There is also an improvement in "and" and "or" questions. This

Question: Is it true that the train next to a platform and the train near the snow are in the same color?
Ground Truth Answer: no

CodeVQA

```
images = open_images("ImageSet7.jpg")
platform_image = find_matching_image(images,
"train next to a platform")
snow_image = find_matching_image(images,
"train near the snow")
platform_train_color = query(platform_image,
"What color is the train?")
snow_train_color = query(snow_image, "What
color is the train?")
if platform_train_color == snow_train_color:
    answer = "yes"
else:
    answer = "no"
```

Answer: no

Few-shot PnP-VQA

Context:
Image 1: A train sitting at a large yellow station. A train is pulling into the train station at a platform. A yellow train is parked at a train dock.
Image 2: A locomotive moving down a snowy road. Train train red box on train train train up train cargo train red train train red. A train traveling through a snowy city next to a red light.
===
Q: Is it true that the train next to a platform and the train near the snow are in the same color?
A:

Answer: yes

Figure 2: **Qualitative Comparison**. CodeVQA correctly answers this question from COVR by breaking it into simpler questions, answering each separately, and comparing the answers. Few-shot PnP-VQA answers incorrectly, even though the captions contain the necessary information. (Note that CodeVQA also generates captions, which are not shown here.)

Retrieval Method	Few-shot PnP-VQA	CodeVQA
text-davinci-003		
Random	48.15	49.9
Embedding	49.4	52.5
code-davinci-002		
Random	49.5	50.7
Embedding	52.1	55.3

Table 2: **Comparing Example Retrieval Techniques** on 2000 GQA validation examples. Italicized GPT model name denotes the model used as the question-answering LM.

improvement could be related to the recent finding that LMs benefit from converting multi-hop into single-hop questions (Press et al., 2022).[3]

	Number of images				
	1	2	3	4	5
# of Instances	12	915	828	696	4440
Few-shot PnP-VQA	91.7	51.5	48.3	47.0	46.9
CodeVQA	75.0	53.3	48.7	53.2	53.4

Table 3: **Accuracy by number of images per instance** on COVR validation set.

We analyzed sources of error in CodeVQA on 100 examples in the COVR validation set for which CodeVQA answered incorrectly: irrelevant captions (31%), mistake in find_matching_image (12%), program generation error (14%), question-answering error (25%), predicted answer could be considered correct (14%), ground-truth is unclear/incorrect (16%), and numerical error (1%). Note that these categories are not mutually exclusive, and 13 of the 100 examples were marked with multiple categories. Thus, more errors are due to execution of the modules than program generation.

[3]Accuracy on this kind of question can also be improved by improving the LM. For instance, using text-davinci-003 as the LM for QA closes the gap between Few-shot PnP-VQA and CodeVQA on "and" questions in GQA.

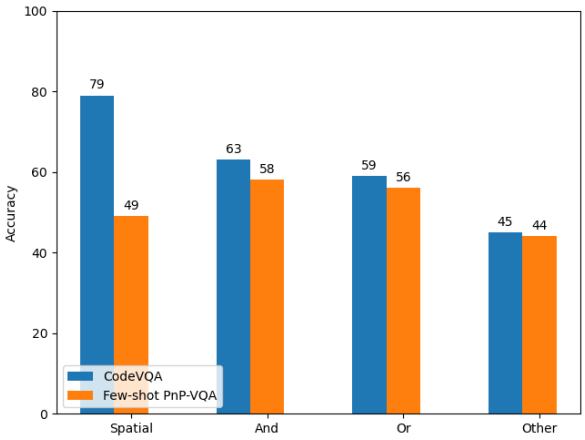

Figure 3: **Accuracy by question type in GQA test set.** CodeVQA (blue) outperforms Few-shot PnP-VQA (orange) on the spatial, and, or questions. "Spatial" refers to questions focusing on left/right or top/bottom relations or object positions.

5 Conclusion

In this paper, we have introduced a framework for modular few-shot VQA. Our approach prompts an LM to generate a Python program that invokes pre-trained visual modules and composes the outputs of these modules to predict the answer. Unlike previous modular VQA techniques, this framework does not require (re-)training modules or a parser. Also, obtaining interpretable module outputs from previous modular approaches is nontrivial (Subramanian et al., 2020), whereas in our approach the modules are frozen and thus interpretable. CodeVQA can also be viewed as expanding pipelined systems (Zeng et al., 2022) to the full expression of code. Our approach exhibits empirical gains, motivating future work on modular few-shot VQA.

6 Limitations

While the initial results are promising, the accuracy of our method remains lower than human VQA accuracy and models finetuned on the VQA datasets,

which suggests that there may still be substantial progress that must be made before few-shot VQA methods with code synthesis are useful for practical real world applications. Also, further work is needed on extending the framework to additional primitives, as the results in Appendix G show that doing so does not always lead to improvements over the baseline method. Another limitation of our approach is that it relies on large capable LMs, which may be restricted in use due to compute requirements or cost (e.g. via available APIs). We also focus in this work on benchmarking VQA capabilities with English as the primary language – future work may extend this to other languages via multilingual LMs.

7 Acknowledgements

We thank the members of the Berkeley NLP group (especially Eric Wallace), Grace Luo, and the anonymous reviewers for feedback on earlier drafts of this paper. We are grateful to Ben Bogin and Shivanshu Gupta for their assistance in evaluating `CodeVQA` and Few-shot PnP-VQA on the private COVR test set. SS, MN, and TD were supported in part by the DoD, including an NDSEG fellowship (for SS) and DARPA's LwLL, PTG, and/or SemaFor programs, by the NSF, and/or by the Berkeley Artificial Intelligence Research (BAIR) industrial alliance program.

References

Sandhini Agarwal, Gretchen Krueger, Jack Clark, Alec Radford, Jong Wook Kim, and Miles Brundage. 2021. Evaluating clip: Towards characterization of broader capabilities and downstream implications. *ArXiv*, abs/2108.02818.

Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andy Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. 2022. Flamingo: a visual language model for few-shot learning. *ArXiv*, abs/2204.14198.

Jacob Andreas, Marcus Rohrbach, Trevor Darrell, and Dan Klein. 2016. Learning to compose neural networks for question answering. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1545–1554, San Diego, California. Association for Computational Linguistics.

Ben Bogin, Shivanshu Gupta, Matt Gardner, and Jonathan Berant. 2021. Covr: A test-bed for visually grounded compositional generalization with real images. In *Conference on Empirical Methods in Natural Language Processing*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Igor Babuschkin, S. Arun Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code. *ArXiv*, abs/2107.03374.

Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. 2023. Binding language models in symbolic languages. In *The Eleventh International Conference on Learning Representations*.

Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. 2017. Making the v in vqa matter: Elevating the role of image understanding in visual question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6904–6913.

Tanmay Gupta and Aniruddha Kembhavi. 2023. Visual programming: Compositional visual reasoning without training. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Ronghang Hu, Jacob Andreas, Marcus Rohrbach, Trevor Darrell, and Kate Saenko. 2017. Learning to reason: End-to-end module networks for visual question answering. In *Proceedings of the IEEE international conference on computer vision*, pages 804–813.

Drew A. Hudson and Christopher D. Manning. 2019. Gqa: A new dataset for real-world visual reasoning and compositional question answering. *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6693–6702.

Woojeong Jin, Yu Cheng, Yelong Shen, Weizhu Chen, and Xiang Ren. 2021. A good prompt is worth millions of parameters: Low-resource prompt-based learning for vision-language models. *ArXiv*, abs/2110.08484.

Ranjay Krishna, Yuke Zhu, Oliver Groth, Justin Johnson, Kenji Hata, Joshua Kravitz, Stephanie Chen, Yannis Kalantidis, Li-Jia Li, David A. Shamma, Michael S. Bernstein, and Li Fei-Fei. 2016. Visual genome: Connecting language and vision using crowdsourced dense image annotations. *International Journal of Computer Vision*, 123:32–73.

Jayant Krishnamurthy and Thomas Kollar. 2013. Jointly learning to parse and perceive: Connecting natural language to the physical world. *Transactions of the Association for Computational Linguistics*, 1:193–206.

Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. BLIP-2: bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML*.

Junnan Li, Dongxu Li, Caiming Xiong, and Steven C. H. Hoi. 2022. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *ICML*.

Junnan Li, Ramprasaath R. Selvaraju, Akhilesh Deepak Gotmare, Shafiq R. Joty, Caiming Xiong, and Steven C. H. Hoi. 2021. Align before fuse: Vision and language representation learning with momentum distillation. In *Neural Information Processing Systems*.

Liunian Harold Li, Mark Yatskar, Da Yin, Cho-Jui Hsieh, and Kai-Wei Chang. 2019. Visualbert: A simple and performant baseline for vision and language. *ArXiv*, abs/1908.03557.

J. Liang, Wenlong Huang, F. Xia, Peng Xu, Karol Hausman, Brian Ichter, Peter R. Florence, and Andy Zeng. 2022. Code as policies: Language model programs for embodied control. *ArXiv*, abs/2209.07753.

Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, et al. 2023. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*.

Kenneth Marino, Mohammad Rastegari, Ali Farhadi, and Roozbeh Mottaghi. 2019. Ok-vqa: A visual question answering benchmark requiring external knowledge. *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3190–3199.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke E. Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Francis Christiano, Jan Leike, and Ryan J. Lowe. 2022. Training language models to follow instructions with human feedback. *ArXiv*, abs/2203.02155.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. 2022. Measuring and narrowing the compositionality gap in language models. *ArXiv*, abs/2210.03350.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pages 8748–8763. PMLR.

Candace Ross, Boris Katz, and Andrei Barbu. 2020. Measuring social biases in grounded vision and language embeddings. In *North American Chapter of the Association for Computational Linguistics*.

Raeid Saqur and Karthik Narasimhan. 2020. Multimodal graph networks for compositional generalization in visual question answering. In *Neural Information Processing Systems*.

Ramprasaath R. Selvaraju, Abhishek Das, Ramakrishna Vedantam, Michael Cogswell, Devi Parikh, and Dhruv Batra. 2016. Grad-cam: Visual explanations from deep networks via gradient-based localization. *International Journal of Computer Vision*, 128:336–359.

Sanjay Subramanian, Ben Bogin, Nitish Gupta, Tomer Wolfson, Sameer Singh, Jonathan Berant, and Matt Gardner. 2020. Obtaining faithful interpretations from compositional neural networks. In *Annual Meeting of the Association for Computational Linguistics*.

Sanjay Subramanian, William Merrill, Trevor Darrell, Matt Gardner, Sameer Singh, and Anna Rohrbach. 2022. ReCLIP: A strong zero-shot baseline for referring expression comprehension. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5198–5215, Dublin, Ireland. Association for Computational Linguistics.

Alane Suhr, Stephanie Zhou, Ally Zhang, Iris Zhang, Huajun Bai, and Yoav Artzi. 2019. A corpus for reasoning about natural language grounded in photographs. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 6418–6428, Florence, Italy. Association for Computational Linguistics.

Dídac Surís, Sachit Menon, and Carl Vondrick. 2023. Vipergpt: Visual inference via python execution for reasoning. *arXiv preprint arXiv:2303.08128*.

Anthony Meng Huat Tiong, Junnan Li, Boyang Li, Silvio Savarese, and Steven CH Hoi. 2022. Plug-and-play vqa: Zero-shot vqa by conjoining large pre-trained models with zero training. *Findings of ACL: EMNLP*.

Zhenhailong Wang, Manling Li, Ruochen Xu, Luowei Zhou, Jie Lei, Xudong Lin, Shuohang Wang, Ziyi Yang, Chenguang Zhu, Derek Hoiem, Shih-Fu Chang, Mohit Bansal, and Heng Ji. 2022. Language models with image descriptors are strong few-shot video-language learners. *ArXiv*, abs/2205.10747.

Zhengyuan Yang, Zhe Gan, Jianfeng Wang, Xiaowei Hu, Yumao Lu, Zicheng Liu, and Lijuan Wang. 2021. An empirical study of gpt-3 for few-shot knowledge-based vqa. In *AAAI Conference on Artificial Intelligence*.

Mark Yatskar, Luke Zettlemoyer, and Ali Farhadi. 2016. Situation recognition: Visual semantic role labeling for image understanding. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5534–5542.

Andy Zeng, Adrian Wong, Stefan Welker, Krzysztof Choromanski, Federico Tombari, Aveek Purohit, Michael Ryoo, Vikas Sindhwani, Johnny Lee, Vincent Vanhoucke, et al. 2022. Socratic models: Composing zero-shot multimodal reasoning with language. *arXiv preprint arXiv:2204.00598*.

Xiaohua Zhai, Xiao Wang, Basil Mustafa, Andreas Steiner, Daniel Keysers, Alexander Kolesnikov, and Lucas Beyer. 2022. Lit: Zero-shot transfer with locked-image text tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18123–18133.

Pengchuan Zhang, Xiujun Li, Xiaowei Hu, Jianwei Yang, Lei Zhang, Lijuan Wang, Yejin Choi, and Jianfeng Gao. 2021. Vinvl: Revisiting visual representations in vision-language models. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5575–5584.

A Code generation prompts

A.1 GQA

The preamble of the prompt (gray)–containing the instruction, constants, import statements, and API documentation–and a single in-context example are shown below (question in green, program highlighted). In our main GQA experiments, 12 in-context examples are used for each evaluation example.

```
"""Write Python code to answer the questions about each image."""
# Global constants
# min x coordinate LEFT = 0
# min y coordinate BOTTOM = 0 # max x coordinate RIGHT = 24 # max y coordinate TOP = 24 from PIL import Image from utils import open_images, query, find_matching_image, get_pos
"""
API Reference:
open_image(path: str) -> Image - opens the image at the path and returns it as an Image object
query(img: Image, question: str) -> str - queries the image returns an answer to the question
get_pos(img: Image, object: str) -> (float, float) - returns the position of the object in the image """
# Image 1: Does the bench look silver and metallic?
img = open_image("Image1.jpg")
is_silver = query(img, "Does the bench look silver and metallic?")
is_metallic = query(img, "Does the bench look metallic?")
if is_silver == "yes" and is_metallic == "yes":
    answer = "yes"
else:
    answer = "no"
```

A.2 COVR

The preamble of the prompt (gray)–containing the instruction, constants, import statements, and API documentation–and a single in-context example (question in green, program highlighted) are shown below. In our COVR experiments, 6 in-context examples are used for each evaluation example.

```
"""Write Python code to answer the questions about each image."""
# Global constants
# min x coordinate LEFT = 0
# min y coordinate BOTTOM = 0 # max x coordinate RIGHT = 24 # max y coordinate TOP = 24 from PIL import Image from utils import open_images, query, find_matching_image, get_pos
"""
API Reference:
open_image(path: str) -> List[Image] - opens the images in the given directory and returns them in a list of Image objects
query(img: Image, question: str) -> str - queries the region of the image in the given coordinates and returns an answer
find_matching_image(images: List[Image], text: str) -> Image - returns the image that best matches the text
get_pos(img: Image, object: str) -> (float, float) - returns the position of the object in the image """
# Image Set 1: Is it true that there are more ladies that are wearing black shirt than men that are wearing black shirt?
images = open_images("ImageSet1.jpg")
ladies_total = 0
men_total = 0
for image in images:
    ladies_exist = query(image, "Is there a lady?")
    if ladies_exist == "yes":
        ladies_count = int(query(image, "How many ladies are wearing black shirt?"))
        ladies_total += ladies_count
    man_exist = query(image, "Is there a man?")
    if men_exist == "yes":
        men_count = int(query(image, "How many men are wearing black shirt?"))
        men_total += men_count
if ladies_total > men_total:
    answer = "yes"
else:
    answer = "no"
```

B GradCAM

Our computation of GradCAM follows prior work that uses vision transformers (Tiong et al., 2022; Li et al., 2021). We are given a question with tokens $q_1, ..., q_T$ and an image that is tokenized into $K \times K$ patches. We use layer $L = 6$ to compute GradCAM, following Tiong et al. (2022). We compute a GradCAM map for each token as follows. Let $C \in \mathbb{R}^{T \times K^2}$ be the cross-attention map from layer L. Let $G \in \mathbb{R}^{T \times K^2}$ be the gradient of the image-text matching score with respect to C. Then the GradCAM map for token i is given by the ith row of $C \bigodot ReLU(G))$, where $\bigodot$ denotes elementwise multiplication. As stated in Section 3.1, for the `query` primitive, we take the average GradCAM map across all question tokens, whereas for the `get_pos` primitive, we take the average GradCAM map across the input text tokens (which are part of the question tokens).

C Implementation Details

To generate captions for in-context examples in each dataset, we run steps $1 - 4$ for each of the 50 questions in the database of in-context examples. For GQA experiments, we use $C = 7$ captions per image, and for COVR experiments, where each question is associated with multiple images, we use $C = 3$ captions per image.[4] We use $C = 7$ captions for the NLVR2 dataset. Each reported accuracy result represents a single evaluation run over the corresponding evaluation set. For NLVR2 and some instances of COVR, the text input is a statement (to be classified as True/False). We convert each such statement to a question by adding the prefix "Is it true that" to it and converting the answer to "yes"/"no." We use question embeddings to select 12 examples for GQA and 6 examples for COVR and NLVR2.

D Details on Baselines

FewVLM randomly samples 16 few-shot examples for GQA. VisProg runs the program generation and execution pipeline five times, and for each iteration randomly samples 24 few-shot examples for GQA and 16 few-shot examples for NLVR2. ViperGPT uses 8 few-shot examples for GQA. VisProg uses `text-davinci-002` or `text-davinci-003` for code generation (according to the code release), while ViperGPT uses `code-davinci-002`.

E Qualitative Comparisons

We include qualitative comparisons of our method CodeVQA to the baseline Few-shot PnP-VQA (text-davinci-003) in Fig 5. In all the instances, we can see that PnP-VQA produces captions that are irrelevant to the question, resulting in incorrect answers. On the other hand, `CodeVQA` breaks down the question into a Python code block. `CodeVQA` uses if-else conditions along with the predefined visual modules `get_pos(image, text)`

[4] We chose this number of captions to be the maximum possible subject to the number of shots and the context size of the `davinci` model, which we used as our question-answering LM in preliminary experiments.

Model	GQA Shots	GQA Val Sample	GQA Testdev	COVR Shots	COVR Val Sample	COVR Val	COVR Test
Few-shot PnP-VQA w/ `text-davinci-003`	12	49.4	44.9	6	51.4	–	–
CodeVQA (ours) w/ `text-davinci-003`	12	52.5	46.8	6	54.4	–	–
Few-shot PnP-VQA w/ `code-davinci-002`	12	52.1	46.6	6	49.0	47.8	45.8
CodeVQA (ours) w/ `code-davinci-002`	12	**55.3**	**49.0**	6	**54.5**	**52.9**	**50.7**

Table 4: **Validation and test results on GQA and COVR.** OpenAI model name (`text-davinci-003` or `code-davinci-002`) denotes which model was used as the question-answering model. GQA validation sample contains 2000 examples from the GQA validation set. COVR validation sample contains 1000 examples from the COVR non-paraphrased validation set. Highest scores on are in **bold**.

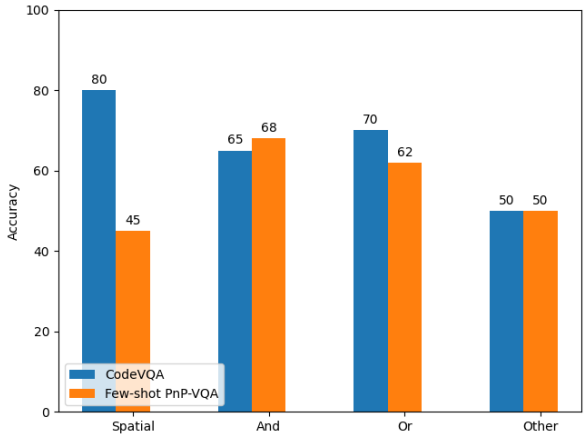

Figure 4: **Accuracy by question type in 2000 GQA validation examples.** CodeVQA (blue) outperforms Few-shot PnP-VQA (orange) on the spatial and or questions. "Spatial" refers to questions focusing on left/right or top/bottom relations or object positions.

and `query(image, text)` to focus on the right regions of the image, arriving at the correct answer in an explainable fashion.

Fig. 6 shows two examples from the NLVR-2 dataset where our method CodeVQA answers the questions correctly. In the first example, it queries each of the images for the count of the pandas, and answers the question correctly based on that. In the second example, our method breaks the question down into three simpler queries and an `if-else` statement to arrive at the correct answer.

Fig. 7 shows the correct results of our method on complex multireference questions in the COVR dataset. CodeVQA is able to break down the logic to obtain the counts of images with a cake on a white plate and images with a lemon on a white plate and then evaluates if the two counts are the same.

In the second more complex example, our method uses `for` loops and complex `if-else` logic to first locate the images that satisfy the criterion, *"pillows on a couch near a table"* and *"pillows on a couch near a bed"* to count the individual occurrences.

F Additional Quantitative Results

Table 4 shows results on validation sets and compares the accuracies of CodeVQA and Few-shot PnP-VQA when using `code-davinci-002` and `text-davinci-003` as the question-answering LM.

Table 5 shows how the accuracies of CodeVQA and Few-shot PnP-VQA vary with the number of shots in the prompt. Figure 4 shows the breakdown of accuracy by question type for 2000 GQA validation examples, which we used for initial experimentation (similar to Figure 3 but on validation examples). We note that on this sample, Few-shot PnP-VQA has an advantage on "and" questions.

Method	Number of shots 8	12	16
text-davinci-003			
Few-shot PnP-VQA	48.3	49.4	49.5
CodeVQA	52.8	52.5	52.7
code-davinci-002			
Few-shot PnP-VQA	50.6	52.1	51.2
CodeVQA	55.1	55.3	55.4

Table 5: Accuracy with different numbers of shots on 2000 GQA validation examples.

Figure 5: **GQA Results**. We show example results from the GQA dataset where our method CodeVQA outperforms the baseline Few-Shot PnP-VQA.

G Experiments with Additional Primitives

We also experiment with two other primitives, on datasets involving counting objects or knowledge retrieval:

`find_object(image, object_description)` This function returns a set of references to objects in the image that match the given description, and we use it for counting objects. We implement this function using Grounding DINO (Liu et al., 2023), which is an open-vocabulary object detector that is also trained on referring expression comprehension.

We evaluate this primitive on the VQAv2 dataset (Goyal et al., 2017), for which we use only this primitive and `query`, as well as the COVR and NLVR2 datasets. We used 12 in-context examples for the VQAv2 dataset. Table 6 shows the results indicating that using this module for counting rather than `query` yields mixed results. Qualitatively, we observe a few reasons for errors in the `find_object` version. First, the object detector is not always accurate (e.g. finding "person holding a banana" when there is a person but no banana). Second, our program may omit key details from the question (e.g. for "How many boats have people in them?" the program counts the number of boats overall). Third, our program may invoke the detector when it is ill-suited to the question (e.g. "How many blades of grass surround the fire hydrant?"). On the other hand, captions often convey the number of objects when the number is small, which is very common in these datasets, so `query` can be effective on counting.

`knowledge_query(question)` This function returns the answer to a question based on world knowledge (e.g. "Which football team has won the most Super Bowls?"). We implement this function using the same LM that is used for `query`. In order to better match the format of the OK-VQA dataset, we add a large negative bias to the logits of the following tokens to prevent the LM from

Question: Is it true that there are four pandas?

Code:

```
panda_count = 0
for image in images:
        panda_count += int(query(image, "How many pandas are there?"))
if panda_count == 4:
        answer = "yes"
else:
        answer = "no”
```

Answer: Yes

Question: Is it true that the left image shows laptops in horizontal rows of three and includes rows of open laptops and rows of closed laptops?

Code:

```
images = open_images("ImageSet7.jpg")
rows_of_three = query(images[0], "Are the laptops in horizontal rows of three?") == "yes”
open_laptops = query(images[0], "Are there rows of open laptops?") == "yes”
closed_laptops = query(images[0], "Are there rows of closed laptops?") == "yes”
if rows_of_three and open_laptops and closed_laptops:
        answer = "yes”
else:
        answer = "no”
```

Answer: No

Figure 6: **NLVR2 Results**. We show example results from the NLVR-2 dataset of our method CodeVQA.

generating them: hyphens, “to”, and °. This choice was made based on preliminary experiments on the OK-VQA dataset.

We evaluate this primitive on the OK-VQA dataset (Marino et al., 2019), for which we use only this primitive and query. For CodeVQA and Few-shot VQA, we used 7 in-context examples to be consistent with the OK-VQA results of ViperGPT (Surís et al., 2023). Table 7 provides the results, showing that for questions involving both visual information and general knowledge, breaking down the questions in this way does not lead to improved accuracy.

For both VQAv2 and OK-VQA, we use the standard evaluation method associated with the VQAv2 dataset, which takes into account the set of ground-truth answers for each question. The Flamingo (Alayrac et al., 2022) results that we report on both datasets used 32 in-context examples.

H Licenses and Other Dataset Details

GQA is licensed under the CC-BY-4.0 license (https://creativecommons.org/licenses/by/4.0/). The COVR repository (https://github.com/benbogin/covr-dataset) is licensed under an MIT license (though imSitu images may not be licensed). The text in both datasets is written in English. The annotations in NLVR2 are licensed under CC-BY-4.0, but the images in the dataset are not licensed. The annotations in

Question: What amount of pictures show girls wearing a skirt and holding a racket?

Code:

```
images = open_images("ImageSet7.jpg")
count = 0
for image in images:
    girl_exists = query(image, "Is there a girl wearing a skirt?")
    if girl_exists == "yes":
        holding_racket = query(image, "Is there a girl holding a racket?")
        if holding_racket == "yes":
            count += 1
answer = count
```

Answer: 1

Question: Are there the same number of images that have a cake on a white plate as there are images that have a lemon on a white plate?

Code:

```
images = open_images("ImageSet7.jpg")
cake_count = 0
lemon_count = 0
for image in images:
    cake_exists = query(image, "Is there a cake on a white plate?")
    lemon_exists = query(image, "Is there a lemon on a white plate?")
    if cake_exists == "yes":
        cake_count += 1
    if lemon_exists == "yes":
        lemon_count += 1
if cake_count == lemon_count:
    answer = "yes"
else:
    answer = "no”
```

Answer: No

Figure 7: **COVR Results**. We show results on the COVR dataset where our method correctly answers the question by referencing all the images.

	VQAv2	COVR	NLVR2
Zero-shot			
BLIP-v2	65.0	–	–
Few-shot			
Flamingo	67.6‡	–	–
Few-shot PnP-VQA	66.84	47.8	63.4
CodeVQA	–	52.9	64.0
CodeVQA w/ find_object	66.63	52.9	66.0

Table 6: Results with find_object used for counting objects on VQAv2 (a random sample of 4000 examples from validation set), COVR (validation), and NLVR2 (test-public). ‡indicates a result on the full VQAv2 test-dev set, which may not be directly comparable with our results on a sample of the validation set.

	OK-VQA
Zero-shot	
BLIP-v2	45.9
Few-shot	
Flamingo	57.8
ViperGPT	51.9
Few-shot PnP-VQA	54.1
CodeVQA w/ knowledge_query	53.5

Table 7: Results with knowledge_query on the OK-VQA validation set.

VQAv2 are licensed under CC-BY-4.0.

The testdev set of GQA contains 12578 instances. The test set of COVR contains 7024 instances. The validation set of COVR contains 6891 instances. The public test set of NLVR2 contains 6967 instances. The validation set of OK-VQA contains 5046 instances. For VQAv2, we evaluate on a random sample of 4000 examples from the validation set.

During the development and intermediate evaluations of our method, we evaluated on a random sample of 200 training examples and a random sample of 2000 validation examples from GQA, a random sample of 200 training examples and the validation set from COVR, a random sample of 2000 training examples from NLVR2, a random sample of 1200 training examples from OK-VQA, and a random sample of 2200 training examples from VQAv2.

I Ethics and Impact Statement

One goal of our work is to decrease the need for (re-)training VQA systems. Achieving this goal would mean a decrease in carbon emissions from training models. However, our approach also has a high inference cost, given the use of large language models. A decision to employ our approach should take into consideration this computational cost and the associated environmental impact.

Another potential positive impact of our approach is improved interpretability via the generated programs. These programs offer to people familiar with Python a record of which visual tasks the system uses for a given question and how the system combines the outputs of these tasks to predict the answer.

Our system relies on pre-trained vision-language models to predict answers to visual questions. Prior work (Ross et al., 2020; Agarwal et al., 2021) has found evidence of social biases in vision-language models trained on image-captions. Therefore, our system may exhibit these biases as well. Practitioners should be aware of this risk and ideally should take steps to mitigate this risk when they consider deploying this system in ways that can impact human lives.